RAG Hallucination Detector

Project summary · trained on RAGTruth · scikit-learn + SQLite + Streamlit

1. What the project does

The user gives two pieces of text: the **context** (the passages a RAG system retrieved) and the **response** (the LLM's answer). The detector checks every sentence of the response against the context. It shades unsupported sentences red, underlines words that never appear in the context, and returns a risk score and a verdict for the whole answer. Everything runs from one notebook (Run All), with a Streamlit app on localhost and a SQLite database behind it.

2. Models used

Role	Model	Notes
Detector (the judge)	Gradient Boosting: scikit-learn HistGradientBoostingClassifier	300 iterations, learning rate 0.05, balanced class weights. Chosen model.
Detector candidates	Logistic Regression, Random Forest	Trained and compared; lost on dev F1.
Answer generator	gemini-3.5-flash-lite (default), gemini-3.8-flash	Ask Gemini mode only. Gemini writes answers; it never judges them.
LLMs in the training data	GPT-4, GPT-3.5-turbo, Llama-2 7B/13B/70B chat, Mistral-7B-instruct	The 6 models whose answers RAGTruth annotated.

How the detector is picked: all three classifiers are trained on the RAGTruth train split. The best F1 on a separate **dev** split wins (Gradient Boosting, dev F1 0.392), and the decision threshold is tuned on dev too (0.62 in the latest run). The test split is used only for reporting. The dev split is grouped by source passage, so the 6 answers to one passage never leak across splits.

3. How a response is judged

Training basis: human labels. RAGTruth has 17,790 LLM responses where people marked the exact hallucinated character spans. Each response is split into sentences; a sentence is labelled hallucinated if it overlaps a human-marked span. The model learns to reproduce those judgements.

Evidence: 10 features per sentence, each computed against the context:

Feature	What it measures	Why it signals hallucination
novel_word_ratio, novel_words	Content words never seen in the context	Facts that were not given
novel_numbers, novel_number_ratio	Numbers or dates not in the context	Invented figures (1925 vs 1889)
novel_capitals	Capitalised words not in the context	Invented names or entities
bigram_overlap, trigram_overlap	Share of 2- and 3-word phrases copied from the context	Grounded sentences reuse the source's wording
n_words, position, is_last	Sentence length and position in the answer	Hallucinations cluster in certain spots, e.g. closing 'extra' facts

Decision rule

- Each sentence gets a probability *p* from the model; it is **flagged** if *p* >= 0.62.
- **Risk score** = the highest sentence probability in the answer.
- **Verdict** = "Likely hallucinated" if the risk score >= 0.62, otherwise "Looks grounded". One unsupported sentence is enough to distrust the whole answer.

4. Results (held-out test set, latest run)

Model	Level	Precision	Recall	F1	ROC-AUC
Logistic Regression	Sentence	0.211	0.522	0.301	0.771
Random Forest	Sentence	0.237	0.535	0.328	0.784
Gradient Boosting	Sentence	0.235	0.582	0.334	0.797
Logistic Regression	Response	0.526	0.830	0.644	0.781
Random Forest	Response	0.539	0.806	0.646	0.793
Gradient Boosting	Response	0.538	0.836	0.654	0.800

- At response level ("should I trust this answer?") it catches about **84%** of hallucinated answers, but roughly half of its alarms are false positives.
- Sentence-level F1 is low partly because only about 8% of test sentences are hallucinated.
- Response F1 by task: **Data-to-text 0.78, QA 0.53, Summary 0.39**. On Data-to-text it flags almost everything because the context is JSON, so any rewording looks like new words.

5. Limitations and next steps

The features compare **words**, not **meaning**:

- **False alarms:** a correct paraphrase in different words looks novel and gets flagged.
- **Missed hallucinations:** a contradiction that reuses the context's own words ("A beat B" vs "B beat A") looks grounded.

Next steps: add an NLI (entailment) score as a feature, or train a transformer token classifier (DeBERTa or ModernBERT on [context] [SEP] [response]) to catch contradictions and accept paraphrases.

6. Project at a glance

Piece	Details
Notebook	RAGTruth_Hallucination_Detector.ipynb: Run All installs, asks for a Gemini key once, downloads data, runs EDA, trains 3 models, launches the app.
Backend	backend.py: download, SQLite, features, training, inference, Gemini calls with retry/fallback.
App	app.py (Streamlit, 4 tabs): Detector (Write my own / Ask Gemini / RAGTruth example), Dataset, Model Performance, History.
Database	ragtruth.db: sources, responses, runs (every training result), predictions (every detection).
Tests	python backend.py (self-check) and python test_app.py (7 end-to-end UI tests).